



30. Oversampling and Undersampling: Handling Imbalanced Data

There is actually a problem that is even worse than missing data.

Previous Section:

 [29. One-vs-One and One-vs-All: Multi-Class Classification](#)

Contents

- [1. Random Sampling - A probability concept](#)
- [2. Undersampling — Favor the minority, reduce the majority](#)
- [3. Oversampling — Increase the minority, keeping the majority intact](#)
- [4. Random Sampling with Python](#)
- [5. Model Reliability — Accurate, but not applicable?](#)

[Summary](#)

[References](#)

In this section, I will guide through another important concept that is often overlooked. And the problem here lies with the data itself.

In a section far more previous than this, we talked about Data Preparation. Fixing missing values. Removing noise. Handling uncorrelated features. Splitting data into training and testing sets. All of that sounds clean, controlled, almost reassuring.

But here's the uncomfortable truth: Even after all of that, the data can still betray you. Not because it is missing. Not because it is messy. But because it is

unbalanced.

Imagine this: You are building a model to detect fraud. Out of 10,000 transactions, 9,900 are normal, 100 are fraud.

Now ask yourself a simple question: If a model just predicts "normal" every single time, what is its accuracy? 99% → Sounds impressive. But in reality? It is completely useless. Because it fails at the one thing that actually matters.

This is the hidden danger of **imbalanced data**. The model doesn't learn the problem. It learns the majority. It becomes biased. It becomes overconfident. And worst of all, it gives you the illusion of performance.

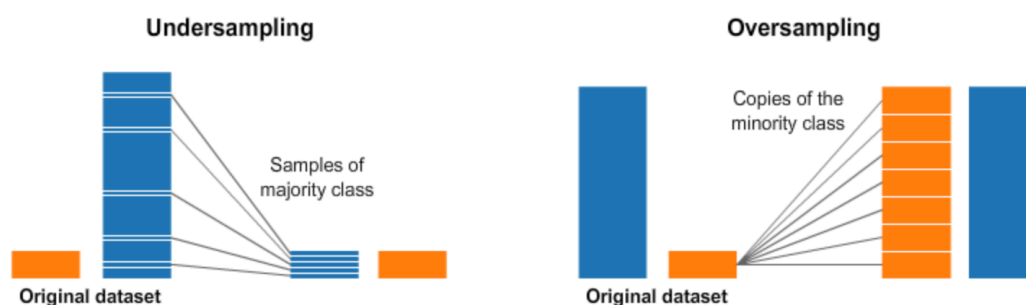
And it doesn't stop there. Even the way we evaluate models can mislead us. A single train-test split might accidentally favor one class more than another. Some rare patterns might not even appear in the test set at all. So the model looks good until it meets reality. And then? It fails.

So now we face two deeper questions:

- How do we make the data more balanced?
- And how do we evaluate models more reliably?

This is where techniques like **Oversampling**, **Undersampling**, and **Cross-Validation** come into play. Not as optional tricks. But as necessary tools to make Machine Learning actually work.

(The following image is a reference to introduce Undersampling and Oversampling)



And in the next section... we start with something simple. Something fundamental. A concept that quietly powers everything behind these

techniques: **Random Sampling**.

1. Random Sampling - A probability concept

(I am thinking of naming this section: "What is an imbalanced dataset?", but I ended up with this name instead)

Let's talk about something that quietly breaks a lot of Machine Learning models. Not the algorithm. Not the math. But the **distribution of the data**.

An imbalanced dataset occurs when one class contains significantly more samples than another. At first glance... this doesn't sound like a big problem. But in practice? It changes everything.

Imagine this: You are building a fraud detection system. 9,900 normal transactions → 100 fraudulent ones. What will the model learn? Not fraud. Not patterns. It learns something much simpler: *"Most of the time... things are normal"*. And it predicts "normal" again and again. Accuracy? → Very high; Usefulness? → Almost zero.

The same thing happens in medical diagnosis. Most people are healthy. Only a small portion have a disease → So the model starts thinking: "People are usually fine."; and technically, it's not wrong. In fact, it almost sounds optimistic: *"The world is healing"*.

But from a Machine Learning perspective? That's a disaster. Because the minority class is usually the one we actually care about: Fraud. Diseases. Failures. → The rare events.

So, "unbalanced" and "imbalanced"? They mean the same thing. But in Machine Learning, this imbalance is not just a linguistic detail. It is a **bias built into the data itself**.

Now this brings us to something very fundamental: **Random Sampling**. At its core, Random Sampling is not a Machine Learning trick. It comes from probability. The idea is simple:

Every member of a population has an equal, non-zero chance of being selected.

That's it. No preference. No bias. No hidden rule.

But what does that actually mean? Think of it like this: You have a population of 100 people. And you want to select 20 of them. Now, how do you choose?

There are two ways to think about it.

1. Sequential selection: You pick people one by one.

- First pick → probability = 1 out of 100
- Second pick → probability = 1 out of 99
- Third pick → 1 out of 98
- ...and so on.

Each step reduces the pool. Each person still has a fair chance, at the moment of selection.

2. Simultaneous selection

Now imagine you select all 20 people at once. You don't care about the order. You only care about the group. So the question becomes: "How many different groups of 20 people can we form from 100?" → The answer is: C_{100}^{20}

And each possible group? Has the same probability. So selecting one specific group is: 1 out of C_{100}^{20}

Two different perspectives. Same idea. Fairness.

And this idea is more important than it looks. Because before we fix imbalance. Before we oversample or undersample. We need to understand one thing clearly: ***What does it mean to sample data fairly?*** Because everything we do next is built on top of this foundation.

2. Undersampling — Favor the minority, reduce the majority

Now if imbalance is the problem. One very natural solution appears:

└ ***"What if we just reduce the majority?"***

That idea leads us to **Undersampling**. At its core, Undersampling does something very simple: It reduces the number of instances in the majority class until it matches the size of the minority class.

No complicated math. No deep optimization. Just cutting things down.

But what does that actually mean in practice? It means this: You **keep all the minority samples** and you **select only a subset of the majority samples**. The rest is discarded.

One of the most common approaches is: **Random Undersampling**. And it is exactly what it sounds like. You randomly remove samples from the majority

class. Until both classes are balanced.

Let's make this real. Imagine a dataset: 9,900 normal transactions; 100 fraudulent transactions

After Random Undersampling: Keep all 100 fraud cases → Randomly select 100 normal cases

Final dataset: 100 normal and 100 fraud → Balanced. Clean. Simple.

And suddenly, something interesting happens. The model can no longer rely on: "Everything is normal". Because now? Normal and fraud appear equally often. So the model is forced to **pay attention**. To actually learn patterns. To actually distinguish.

But here's the catch. And it's an important one. When you remove data. You are not just removing "extra" information. You might be removing:

- Rare but important patterns
- Edge cases
- Subtle variations in the majority class

In other words, you might be throwing away knowledge.

So Undersampling is a trade-off:

- **Pros:** Simpler dataset, faster training, reduced bias
- **Cons:** Loss of information, potential underfitting

And this leads to a very important realization: Undersampling doesn't "fix" the data. It reshapes it.

So the real question becomes: *"Are we removing noise... or removing signal?"*. Because depending on the answer, Undersampling can either help your model or quietly make it worse.

3. Oversampling — Increase the minority, keeping the majority intact

Now what if we take the opposite approach? Instead of reducing the majority. We **lift the minority up**. This is where **Oversampling** comes in. The idea is simple:

- Keep the majority class as it is
- Increase the number of minority samples

- Until both classes are balanced

But here's the key question: *"How do we create more minority data... if we don't actually have it?"*

The simplest answer is: We don't create new data. We **reuse what we already have**.

This leads to the most basic technique: **Random Oversampling**. And just like before, the name tells you everything. We randomly duplicate existing minority-class samples until the dataset becomes balanced.

Let's go back to our example: 9,900 normal transactions and 100 fraudulent ones

After Random Oversampling: Keep all 9,900 normal samples. Duplicate fraud samples until we also have 9,900.

Final dataset: 9,900 normal and 9,900 fraud → Balanced. But in a very different way.

Now something interesting happens again. The model can no longer ignore fraud. Because fraud is no longer rare. It appears again and again and again. So the model starts paying attention.

And this solves one major problem: **Bias toward the majority class is reduced**

→ But just like Undersampling had a hidden cost. Oversampling has one too: When you duplicate data, you are not adding new information. You are repeating the same information. So what does the model learn? Not general patterns. But specific points. Repeated many times.

→ This leads to a serious risk: **Overfitting**

The model starts memorizing the minority samples. Instead of learning how to generalize them. And there are two more subtle problems.

1. Larger dataset: By duplicating data, the dataset grows significantly. More storage. Longer training time. Higher computational cost. Sometimes, unnecessarily large.

2. No new patterns: Because sampling is random. You are not introducing any new structure. No variation. No diversity. Just copies. So if the original minority class was limited, the model remains limited. It just sees the same examples more often.

So Oversampling is also a trade-off:

- **Pros:** Keeps all data, reduces class bias

- **Cons:** Overfitting, larger dataset, no new information

And this leads to a deeper realization: Oversampling doesn't create knowledge. It amplifies what already exists.

So the real question becomes: *"Are we helping the model learn... or just making it memorize faster?"*. Because depending on the answer, Oversampling can be powerful, or dangerously misleading.

4. Random Sampling with Python

Now, everything we've discussed so far has been conceptual. Imbalance. Undersampling. Oversampling. But Machine Learning doesn't live in theory. It lives in code.

Here's the interesting part: **Scikit-learn**, the main library we've been using throughout this course. does **not** provide built-in implementations for oversampling and undersampling. Sounds surprising? A little.

But instead of forcing these techniques into Scikit-learn itself, we use something designed specifically for this problem: the **imbalanced-learn (imblearn) library**. Think of it as an extension. Not a replacement, but a companion.

The **imbalanced-learn** library integrates seamlessly with Scikit-learn workflows and provides: Random Undersampling, Random Oversampling, and more advanced techniques like SMOTE. All designed for one purpose: Handling imbalanced datasets properly.

Let's make this real. Imagine again, a fraud detection problem. Two classes: Genuine transactions (majority) and Fraud transactions (minority).

In the real world, fraud is rare, very rare. And this creates a dangerous situation. If we train a model directly on this data. It might learn something like: *"Everything is Genuine."* → And surprisingly? It will still achieve very high accuracy.

But we already know the truth: High accuracy does not mean good performance (*because if so, other metrics like precision or recall would have not existed*). Especially, in imbalanced datasets.

So let's see this in action using a real dataset: **creditcard.csv**

(This dataset is quite large—over 100 MB—so running it in an IDE like PyCharm is recommended.)

- **Step 1 — Understanding the imbalance**

```
# Credit Card Detection
import pandas as pd

df = pd.read_csv('creditcard.csv')

# Count the number of genuine and fraud transactions
print("Number of Genuine transactions:", (df['Class'] == 0).sum())
print("Number of Fraud transactions:", (df['Class'] == 1).sum())
```

Number of Genuine transactions: 284315

Number of Fraud transactions: 492

Take a moment to look at those numbers: 284,315 vs 492 → That's not just imbalance. That's extreme imbalance.

- **Step 2 — Train a model (without fixing anything)**

```
from sklearn.model_selection import train_test_split

# Split data
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)

# Features and target
target = 'Class'
features = df.columns.tolist()
features.remove(target)

# Separate X and y
X_train = train_df[features]
y_train = train_df[target]
```



```
X_test = test_df[features]
y_test = test_df[target]
```

```
from sklearn.linear_model import LogisticRegression

# Train model
model = LogisticRegression(solver='liblinear')
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
```

- **Step 3 — Evaluate carefully**

```
from sklearn.metrics import accuracy_score

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.9989993328885924

At first glance? This looks incredible. Almost perfect. But now... look deeper.

```
from sklearn.metrics import accuracy_score, classification_
report

report = classification_report(y_test, y_pred)
print("Classification Report:\n", report)
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.83	0.53	0.65	98
accuracy			1.00	56962
macro avg	0.91	0.77	0.82	56962
weighted avg	1.00	1.00	1.00	56962

Let's focus on the precision, recall, and f1-score part. This is why we don't rely on accuracy alone. Because accuracy is dominated by the majority class.

Look at the minority class (Fraud): $Recall = 0.53$ → That means: The model only detects **53% of fraud cases**. Almost half of them? Missed.

So the accuracy is pretty good. But wait!! is this model actually doing its job? Because in fraud detection, missing fraud is not a small mistake. It is the mistake.

And now we are ready to fix it. Not by changing the model. But by changing the data.

- **Step 4: Perform Random Undersampling**

Let's actually fix the imbalance. Not by changing the model. But by changing the data itself.

```

# Random Sampling: Handling imbalanced dataset
from imblearn.under_sampling import RandomUnderSampler

X = df[features]
y = df[target]

rus = RandomUnderSampler(random_state=42)

# Fit and resample
X_resampled, y_resampled = rus.fit_resample(X, y)

# Convert to DataFrame
df_resampled = pd.DataFrame(data=X_resampled, columns=features)
df_resampled[target] = y_resampled

# Count again
print("Number of Genuine transactions:", (df_resampled['Class'] == 0).sum())
print("Number of Fraud transactions:", (df_resampled['Class'] == 1).sum())

```

Number of Genuine transactions: 492

Number of Fraud transactions: 492

Let that sink in for a moment. From: **284,315 vs 492** → To: **492 vs 492**.
Perfectly balanced.

*(Here I chose **Undersampling**. Because Oversampling has a very real downside: it makes the dataset much larger. And while “more data” sounds ideal, it can also be a trap.)*

You are not always gaining information. Sometimes, you are just repeating it.

- **Step 5: Train the model again**

Now we take this new dataset, and repeat the process.

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

```

```

# Split data
train_df, test_df = train_test_split(df_resampled, test_size=0.3, random_state=42)

# Separate features and target
X_resampled_train = train_df[features]
y_resampled_train = train_df[target]

X_resampled_test = test_df[features]
y_resampled_test = test_df[target]

# Train model
imbalanced_model = LogisticRegression(solver='liblinear')
imbalanced_model.fit(X_resampled_train, y_resampled_train)

# Predict
y_pred = imbalanced_model.predict(X_resampled_test)

```

```

from sklearn.metrics import accuracy_score

# Evaluate
accuracy2 = accuracy_score(y_resampled_test, y_pred)
print("Accuracy:", accuracy2)

```

Accuracy: 0.8108108108108109

The result is very different → Accuracy ≈ 0.81 → At first glance? This looks worse. Much worse.

But now, look deeper.

```

from sklearn.metrics import classification_report

classification_report2 = classification_report(y_resampled_test, y_pred)
print("Classification Report:\n", classification_report2)

```

Classification Report:

	precision	recall	f1-score	support
0	0.73	1.00	0.84	150
1	1.00	0.62	0.76	146

accuracy			0.81	296
macro avg	0.86	0.81	0.80	296
weighted avg	0.86	0.81	0.80	296

This is where things become... interesting.

- Before: Accuracy $\approx$ 99.9% $\rightarrow$ Fraud recall = 0.53
- Now: Accuracy $\approx$ 81% $\rightarrow$ Fraud recall = 0.62

The accuracy dropped. But the model improved. Because now, it is no longer hiding behind the majority class. It is actually trying to **detect fraud**. And notice something important:

Precision for fraud = 1.00 $\rightarrow$ When the model says "this is fraud". It is extremely confident.

So what changed? Not the algorithm. Not the hyperparameters. Just the **data distribution**.

And this reveals one of the most important lessons in Machine Learning:

┃ *"A better dataset... often beats a better model."*

So the result is no longer "perfect". But it is finally **meaningful**.

5. Model Reliability — Accurate, but not applicable?

Now here's a dangerous situation. A model shows **very high accuracy**. And yet, It is completely unreliable. Because in Machine Learning, **Accuracy alone means almost nothing** — especially with imbalanced data.

The Core Problem: A model can achieve high accuracy simply by predicting the majority class: "Most people are healthy"; "Most transactions are genuine" → So it keeps saying: "No problem". And most of the time, it's right. But when it matters? It fails.

Confusion Matrix (Scikit-learn Convention)

In binary classification: Class 1 = Positive → Class 0 = Negative

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

From this, we define Accuracy, Precision, Recall, and F1-Score (*The formulas? You have learned them already*)

A Reality Check: Imagine a Heart Disease model with Accuracy = **88.5%**. Sounds good.

→ But look closer: Precision = 0.30; Recall = 0.07; F1-score = 0.11

That recall means: The model detects only **7% of actual patients**. It misses **93%**.

So what is this model actually good at? Saying people are **fine**. And that explains everything: The model performs very well on the **majority class (No disease)**. But fails on the **minority class (Disease)**

The Hidden Truth - High accuracy can simply mean: "The model is good at predicting the majority." → Not: "The model is useful."



Pro Tip for Binary Classification (also good for Multi-Class Classification):

1. The positive class is labeled 1, and the negative class is labeled 0 (*Class 0 = "No / Not" and Class 1 = "Yes / Positive"*) → **Every single time**
2. Always evaluate: Precision; Recall; F1-score for **each class**, not just overall accuracy.

How to Make a Model Reliable - When accuracy is high but the model feels "wrong", fix the data and evaluation:

- **Random Sampling:** Balance the dataset (undersampling / oversampling)
- **Better Models:** Some models (like Gradient Boosting) handle imbalance better
- **Cross-Validation:** Avoid relying on a single split
- **Better Metrics:** Look beyond accuracy → focus on Recall, F1, ROC-AUC
- **More Data:** Especially for the minority class

A reliable model is not the one with the highest accuracy. It is the one that performs well **when it actually matters**.

Summary

Undersampling and Oversampling are important concepts. What's easy, basic, is often overlooked. In Machine Learning, we often focus on: Models; Algorithms; Performance. But sometimes, the real problem is not the model. It's the **data**.

Imbalanced datasets silently bias the model. They make it look accurate. They make it look confident. But underneath? The model is just favoring the majority.

Undersampling and Oversampling give us two simple ways to respond: Reduce the majority or Increase the minority → Different approaches. Same goal:

Restore balance

But neither is perfect: Undersampling may lose information. Oversampling may cause overfitting. So the question is never: "Which one is better?" → But: "What does this dataset need?"

And beyond sampling, we learned something even more important:

Accuracy alone is never enough. A model is not reliable just because it is "correct" most of the time. It must be correct **where it matters**.

Because in the end, a slightly less accurate model that catches fraud, detects disease, or identifies rare events, is far more valuable than a "perfect" model that misses them all.

References

<https://www.geeksforgeeks.org/machine-learning/handling-imbalanced-data-for-classification/>

<https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/>

<https://www.kaggle.com/code/residentmario/undersampling-and-oversampling-imbalanced-data>

Next Section:

 [31. K-Fold Cross Validation](#)